

The lux Hub Replicator: a Z Specification

Formal model of the single-writer, two-lock replication discipline

July 2026

1 Introduction

An MCP `clear` call once froze an agent for about 38 minutes: the tool sent to the display synchronously, on the agent’s own thread, over a socket with no send timeout, while holding the client send lock. The fix moves every send to the display onto one background worker inside the Hub — the *replicator* — and keeps every agent-facing tool on the Hub’s in-memory store, which it updates and returns from at once.

The worker introduces real concurrency. It is a background thread; it drains a set of changed scenes that several other threads mark; it reads a store that those threads mutate; and it kills and restarts a wedged display while those threads keep running. This document models that concurrency as a finite state machine over a small fixed set of scenes and mutator threads, so that ProB can enumerate every interleaving and either confirm the invariants or produce the exact offending trace.

Five properties must hold across every interleaving:

- I1 — single writer.** Only the worker ever writes to the display connection. No mutator thread holds the client send lock.
- I2 — agent-path liveness.** No mutator operation is ever disabled by display I/O or by the client send lock. A mutator makes progress even while the worker is stuck sending to a dead display.
- I3 — no lock cycle.** No thread ever holds the store lock and the client send lock at the same time. The worker snapshots under the store read lock, releases it, and only then takes the client lock to send.
- I4 — no lost update.** Whenever the system comes to rest with the display up, the display shows the store’s latest state for every scene — including after a reap and respawn, which re-marks every live scene, and including after a cycle that failed and restored its own batch.
- I5 — one display.** The worker never spawns a fresh display while one is still alive; a respawn follows a kill.

Recovery and blanking. The worker’s cycle can fail in three ways, and an emptied scene must disappear from the display; both shape the model. A send raises one of two socket errors — a timeout (`BlockingIOError`) or a dead peer (`OSError`) — and each has its own heal. But the heal itself can fail: an unspawnable display or a refused reconnect raises before the work is re-marked, and any other error inside the cycle escapes the two socket handlers. That third, generic failure must not drop the drained work: the worker restores the exact batch — re-marking its scenes *and* re-setting the cleared flag — and backs off. Every heal of a batch that carried a clear re-marks that clear, because a reconnect to the same display leaves the old scenes on screen and only a fresh blank removes them; blank-then-repaint is idempotent. Finally, a drained scene whose store copy is empty is pushed as a blank into its own recorded frame, so an emptied or departed scene vanishes — unless the batch already carried a clear, which blanked the whole display, in which case the empty scene is skipped rather than blanked twice.

2 Basic Types

[*SCENE*, *MUTATOR*]

SCENE is the set of scenes the Hub can hold; two suffice to exhibit a coalesced batch and a per-scene lost update. *MUTATOR* is the set of threads that change the store: MCP tool calls (`show`, `update`, `clear`) and Hub-side click handlers. Both kinds do the same thing to the store, so one carrier covers both; two mutators exhibit lock contention. Both carriers are bounded by ProB’s `DEFAULT_SETSIZE`.

3 Free Types

$VAL ::= vempty \mid vlive \mid vtorn$

The content of one scene in the store or on the display, abstracted to a token. **vempty**: no elements (a scene that has been cleared or never populated). **vlive**: a committed, consistent scene tree. **vtorn**: the half-updated state a scene passes through while a mutator is inside a multi-step **replace_scene** — the value that, if read without the lock, is the “dictionary changed size during iteration” defect. Real scene structure is elided; what matters is whether a read observes a committed value or a torn one.

$DPROC ::= dup \mid dstuck \mid ddead$

The display process as the worker’s send sees it. **dup**: alive and draining its socket; a send succeeds. **dstuck**: alive but no longer reading, so the send buffer fills and the time-limited send raises **BlockingIOError**. **ddead**: the peer is gone, so the send raises a plain **OSError** (**ECONNRESET**/**EPIPE**).

$LOCK ::= held \mid free$

$FLAG ::= set \mid clear$

Two-valued markers for the worker’s read lock and client lock, and for the cleared and shutting signals, respectively. A dedicated free type is used rather than a built-in boolean so the spec is self-contained for fuzz.

$WPHASE ::=$
 $widle \mid wdrain \mid wblank \mid wproc \mid wsnap \mid wsend \mid$
 $wreap \mid wensure \mid wremark \mid wreconn \mid$
 $wflush \mid wstopped$

The worker’s own program counter. **widle**: waiting on the condition. **wdrain**: woken, about to take the whole changed-scene set. **wblank**: batch drained, about to blank the display if the batch carried a clear. **wproc**: iterating the drained batch. **wsnap**: holding the store read lock, copying one scene’s roots. **wsend**: holding the client lock, sending that copy. **wreap**/**wensure**/**wremark**: the reap-respawn-remark recovery after a send timeout. **wreconn**: the reconnect recovery after a dead peer. **wflush**/**wstopped**: the final bounded flush at shutdown. A generic cycle failure returns the worker straight to **widle** after restoring its batch.

4 State

The state is flat: one schema. Its predicate carries only *structural* coherence — the locks and the snapshot hold at most one thing, and the write lock is held exactly when a scene is mid-replace. These hold in every design, correct or buggy, so they constrain the state space without masking the safety properties. The safety properties (Section 10) are deliberately *absent* here, so that a violating state stays reachable and ProB can find it.

Repl

store : *SCENE* $\rightarrow$ *VAL*
displayed : *SCENE* $\rightarrow$ *VAL*
dirty : $\mathbb{P}$ *SCENE*
batch : $\mathbb{P}$ *SCENE*
drained : $\mathbb{P}$ *SCENE*
cleared : *FLAG*
batchCleared : *FLAG*
snapScene : $\mathbb{P}$ *SCENE*
snapVal : *VAL*
storeLockMut : $\mathbb{P}$ *MUTATOR*
mutScene : $\mathbb{P}$ *SCENE*
readLock : *LOCK*
clientLock : *LOCK*
display : *DPROC*
shutting : *FLAG*
wphase : *WPHASE*

$\#storeLockMut \leq 1$
 $\#snapScene \leq 1$
 $\#mutScene \leq 1$
 $(storeLockMut = \emptyset \Leftrightarrow mutScene = \emptyset)$

store is the Hub’s authoritative copy; **displayed** is what the display is currently showing. **dirty** is the changed-scene set guarded by the worker’s condition, and **cleared** the cleared flag beside it — both live signal state that a mutator sets and the worker resets when it drains. **batch** is the set of scenes still to send this cycle — it shrinks as each is pushed — while **drained** is the whole set the cycle drained, kept unshrunk so a generic failure can restore all of it. The two mirror the shipped **DrainedBatch**: its **scenes** field is immutable (**drained**), and the worker’s loop index over it is what **batch** tracks. The distinction is load-bearing: a generic failure of a batch that carried a clear must re-mark *every* scene it drained, not just the unsent remainder, because the re-marked clear will blank the whole display and every drained scene must then be repainted. **batchCleared** is the clear that *that* drained batch carried: the worker snapshots the cleared flag into **batchCleared** when it drains, resets the live flag, and keeps **batchCleared** for the whole cycle, so both the empty-scene skip and a recovery re-mark can still see that the batch was a clearing one. Modelling the two separately is what makes the recovery re-mark of a consumed clear expressible; a single flag, reset when the blank is pushed, would have forgotten by the time a send failed. **snapScene**/**snapVal** are the one scene the worker has copied out under the read lock and the value it copied. **storeLockMut** names the mutator holding the store write lock (empty or one), and **mutScene** the scene it is mid-replacing. **readLock** and **clientLock** record whether the worker holds the store read lock and the client send lock. **display** is the display process state; **shutting** the clean-shutdown signal; **wphase** the worker’s program counter.

Modelling the store read/write lock as *one* mutually-exclusive slot (**readLock** for the sole reader, **storeLockMut** for the one writer) is sound because there is exactly one reader thread in this design — the worker. A read/write lock exists to allow concurrent readers; with a single reader the only exclusion that can ever fire is reader-versus-writer, which a single slot captures.

5 Initialization

Init

Repl'

$store' = (\lambda s : SCENE \bullet vempty)$
 $displayed' = (\lambda s : SCENE \bullet vempty)$
 $dirty' = \emptyset$
 $batch' = \emptyset$
 $drained' = \emptyset$
 $cleared' = clear$
 $batchCleared' = clear$
 $snapScene' = \emptyset$
 $snapVal' = vempty$
 $storeLockMut' = \emptyset$
 $mutScene' = \emptyset$
 $readLock' = free$
 $clientLock' = free$
 $display' = dup$
 $shutting' = clear$
 $wphase' = widle$

A single initial state: empty store, empty display, nothing dirty, no locks held, the worker idle, the display up.

6 Mutator Operations

A mutator changes the store under the write lock in two steps, so the scene passes through a torn intermediate the worker must never read. The first step takes the lock and begins the replace; the second commits the new value, marks the scene dirty, and releases the lock. Marking dirty is part of the commit: the tool calls `mark_dirty` on the same thread, immediately after `replace_scene` returns and before it yields, so no other thread observes the store between the store write and the mark.

The write lock is taken only when it is free *and* the worker is not mid-snapshot (`readLock = free`). That guard is the store lock's reader-writer exclusion; removing it from the worker's snapshot yields the buggy variant of Section 12.

ReplaceBegin

$\Delta Repl$

$m? : MUTATOR$

$s? : SCENE$

$storeLockMut = \emptyset$
 $readLock = free$
 $storeLockMut' = \{m?\}$
 $mutScene' = \{s?\}$
 $store' = store \oplus \{s? \mapsto vtorn\}$
 $displayed' = displayed$
 $dirty' = dirty$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$
 $wphase' = wphase$

ReplaceCommit

$\Delta Repl$

$m? : MUTATOR$

$m? \in storeLockMut$
 $store' = store \oplus (mutScene \times \{vlive\})$
 $dirty' = dirty \cup mutScene$
 $storeLockMut' = \emptyset$
 $mutScene' = \emptyset$
 $displayed' = displayed$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$
 $wphase' = wphase$

A single scene can also be emptied without a clear — an **update** that strips a scene to no roots, or a session whose disconnect blanks the scenes it alone held. **EmptyScene** sets one scene's store value to **vempty** and marks it dirty, under the same write-lock discipline as a replace. It is what makes a departed scene's blank observable to the worker.

EmptyScene

$\Delta Repl$

$s? : SCENE$

$storeLockMut = \emptyset$
 $readLock = free$
 $store' = store \oplus \{s? \mapsto vempty\}$
 $dirty' = dirty \cup \{s?\}$
 $displayed' = displayed$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$
 $wphase' = wphase$

The **clear** tool empties the whole store under the write lock and sets the cleared flag instead of marking a scene dirty. Modelled as one step, since the torn-read race is already exhibited by **replace**:

ClearStore

$\Delta Repl$

```
storeLockMut =  $\emptyset$ 
readLock = free
store' = ( $\lambda s : SCENE \bullet vempty$ )
cleared' = set
displayed' = displayed
dirty' = dirty
batch' = batch
drained' = drained
batchCleared' = batchCleared
snapScene' = snapScene
snapVal' = snapVal
storeLockMut' = storeLockMut
mutScene' = mutScene
readLock' = readLock
clientLock' = clientLock
display' = display
shutting' = shutting
wphase' = wphase
```

7 The Worker Loop

The worker waits on the condition, wakes when something is dirty or the screen was cleared, drains the whole set, and pushes each scene by copying its current roots under the read lock and sending the copy outside the lock. Coalescing needs no extra mechanism: many marks of one scene add it to the set once, and the worker reads that scene's current store value when it snapshots, so the display receives the newest value, never a half-finished one.

Wake

$\Delta Repl$

```
wphase = widle
shutting = clear
( $dirty \neq \emptyset \vee cleared = set$ )
wphase' = wdrain
store' = store
displayed' = displayed
dirty' = dirty
batch' = batch
drained' = drained
cleared' = cleared
batchCleared' = batchCleared
snapScene' = snapScene
snapVal' = snapVal
storeLockMut' = storeLockMut
mutScene' = mutScene
readLock' = readLock
clientLock' = clientLock
display' = display
shutting' = shutting
```

Drain takes the whole changed-scene set at once, snapshots the cleared flag into **batchCleared** for the duration of the cycle, and resets the live **cleared** flag — exactly as **wait_and_drain** builds an immutable **DrainedBatch** and clears the signal. A clear that arrives after this point re-sets the live flag for the next cycle.

Drain

$\Delta Repl$

$wphase = wdrain$
 $batch' = dirty$
 $drained' = dirty$
 $dirty' = \emptyset$
 $batchCleared' = cleared$
 $cleared' = clear$
 $wphase' = wblank$
 $store' = store$
 $displayed' = displayed$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$

At **wblank** the worker blanks the display first if the batch carried a clear, and only then turns to the scenes. **ApplyClear** pushes the blank; **SkipBlank** is the no-clear case that goes straight to the scenes. **batchCleared** is *not* consumed here: it survives the blank so that the per-scene empty skip and any recovery re-mark can still see that this was a clearing batch.

ApplyClear

$\Delta Repl$

$wphase = wblank$
 $batchCleared = set$
 $displayed' = (\lambda s : SCENE \bullet vempty)$
 $wphase' = wproc$
 $store' = store$
 $dirty' = dirty$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$

SkipBlank

$\Delta Repl$

$wphase = wblank$
 $batchCleared = clear$
 $wphase' = wproc$
 $store' = store$
 $displayed' = displayed$
 $dirty' = dirty$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$

Snap takes the store read lock, picks one scene still in the batch, and copies its current value. The read lock is available only when no mutator holds the write lock ($storeLockMut = \emptyset$); that guard is what stops the worker reading a torn scene. Blank-before-repaint is now enforced by the **wblank** phase, which precedes **wproc**, so **Snap** carries no cleared guard of its own.

Snap

$\Delta Repl$

$wphase = wproc$
 $batch \neq \emptyset$
 $storeLockMut = \emptyset$
 $(\exists s : SCENE \bullet$
 $s \in batch \wedge snapScene' = \{s\} \wedge snapVal' = store\ s)$
 $readLock' = held$
 $wphase' = wsnap$
 $store' = store$
 $displayed' = displayed$
 $dirty' = dirty$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$

From **wsnap** the worker either sends the copy or, when the copy is empty and the batch already blanked the whole display with a clear, skips it. **SendBegin** releases the store read lock and only then takes the client send lock; this release-before-acquire is the whole of I3. Its guard $\neg (snapVal = vempty \wedge batchCleared = set)$ is the send/skip partition: an empty scene under a clear is the one case that does not send.

SendBegin

$\Delta Repl$

$wphase = wsnap$
 $\neg (snapVal = vempty \wedge batchCleared = set)$
 $readLock' = free$
 $clientLock' = held$
 $wphase' = wsend$
 $store' = store$
 $displayed' = displayed$
 $dirty' = dirty$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $display' = display$
 $shutting' = shutting$

SendSkip is the complementary case: the scene is empty and the clear already blanked the whole display, so the worker copied nothing worth sending. It releases the read lock, drops the scene from the batch, and sends nothing — **displayed** is untouched, since the clear's blank already set that scene to **vempty**.

SendSkip

$\Delta Repl$

$wphase = wsnap$
 $snapVal = vempty \wedge batchCleared = set$
 $readLock' = free$
 $batch' = batch \setminus snapScene$
 $drained' = drained$
 $snapScene' = \emptyset$
 $snapVal' = vempty$
 $wphase' = wproc$
 $store' = store$
 $displayed' = displayed$
 $dirty' = dirty$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$

The time-limited send has three outcomes, keyed on the display process. On success the display now shows the copied value and the scene leaves the batch. When the copied value is **vempty** — an emptied or departed scene sent without a clear — this is the blank that makes it vanish from its own frame.

SendOk

$\Delta Repl$

wphase = *wsend*
display = *dup*
clientLock' = *free*
displayed' = *displayed* $\oplus$ (*snapScene* $\times$ {*snapVal*})
batch' = *batch* $\setminus$ *snapScene*
drained' = *drained*
snapScene' = $\emptyset$
snapVal' = *vempty*
wphase' = *wproc*
store' = *store*
dirty' = *dirty*
cleared' = *cleared*
batchCleared' = *batchCleared*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
display' = *display*
shutting' = *shutting*

On a send timeout (**BlockingIOError**) the display is alive but wedged. The send gives up within its time limit and releases the client lock, and the worker turns to the reap-respawn path. The batch is kept: the respawn re-marks every live scene, which re-covers the scene in flight.

SendStuck

$\Delta Repl$

wphase = *wsend*
display = *dstuck*
clientLock' = *free*
wphase' = *wreap*
store' = *store*
displayed' = *displayed*
dirty' = *dirty*
batch' = *batch*
drained' = *drained*
cleared' = *cleared*
batchCleared' = *batchCleared*
snapScene' = *snapScene*
snapVal' = *snapVal*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
display' = *display*
shutting' = *shutting*

On a dead peer (**OSError**) there is nothing to kill; the worker closes the dead descriptor and reconnects.

SendDead

$\Delta Repl$

wphase = *wsend*
display = *ddead*
clientLock' = *free*
wphase' = *wreconn*
store' = *store*
displayed' = *displayed*
dirty' = *dirty*
batch' = *batch*
drained' = *drained*
cleared' = *cleared*
batchCleared' = *batchCleared*
snapScene' = *snapScene*
snapVal' = *snapVal*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
display' = *display*
shutting' = *shutting*

8 Reap, Respawn, Reconnect

A stuck display still answers new connections, so it must be killed before a fresh one is started. **Reap** terminates the wedged process; **Ensure** starts a fresh, empty one; **Remark** re-marks every live scene so the fresh display is repainted from the store's current state, and re-marks the clear if the failed batch carried one. **Ensure** is enabled only from **ddead**: a fresh display is never spawned while one is alive, which is I5.

The clear re-mark is the subtle half of the heal. A cleared scene is empty in the store, so it is absent from the live set **Remark** re-marks; without also re-marking the clear, a reconnect to the same surviving display would leave the old, pre-clear scene on screen forever. Re-marking the clear re-arms the live flag, so the next cycle blanks the display again before repainting; blank-then-repaint is idempotent. The re-mark is expressed against **batchCleared** — the clear this failed batch carried — and merges into whatever the live flag already holds, so a fresh clear that arrived mid-cycle is never lost.

Reap

$\Delta Repl$

wphase = *wreap*
display' = *ddead*
wphase' = *wensure*
store' = *store*
displayed' = *displayed*
dirty' = *dirty*
batch' = *batch*
drained' = *drained*
cleared' = *cleared*
batchCleared' = *batchCleared*
snapScene' = *snapScene*
snapVal' = *snapVal*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
clientLock' = *clientLock*
shutting' = *shutting*

Ensure

$\Delta Repl$

$wphase = wensure$
 $display = ddead$
 $display' = dup$
 $displayed' = (\lambda s : SCENE \bullet vempty)$
 $snapScene' = \emptyset$
 $snapVal' = vempty$
 $wphase' = wremark$
 $store' = store$
 $dirty' = dirty$
 $batch' = batch$
 $drained' = drained$
 $cleared' = cleared$
 $batchCleared' = batchCleared$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $shutting' = shutting$

Remark

$\Delta Repl$

$wphase = wremark$
 $dirty' = dirty \cup \{s : SCENE \mid store\ s = vlive\}$
 $(batchCleared = set \Rightarrow cleared' = set)$
 $(batchCleared = clear \Rightarrow cleared' = cleared)$
 $batch' = \emptyset$
 $drained' = \emptyset$
 $wphase' = wproc$
 $store' = store$
 $displayed' = displayed$
 $batchCleared' = batchCleared$
 $snapScene' = snapScene$
 $snapVal' = snapVal$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $display' = display$
 $shutting' = shutting$

Reconn handles the dead-peer path. A dead peer may come back as a fresh, empty display on reconnect, so this path is modelled honestly: the reconnected display shows nothing, and the worker re-marks every live scene — and the clear, if the batch carried one — exactly as **Ensure/Remark** do after a reap. There is no process to kill, so **Reconn** reconnects, resets the display to empty, and re-marks in one step. I4 holds *because of* that re-mark: the fresh display starts blank, so the system cannot come to rest until every live scene is pushed again.

Reconn

$\Delta Repl$

$wphase = wreconn$
 $display' = dup$
 $displayed' = (\lambda s : SCENE \bullet vempty)$
 $dirty' = dirty \cup \{s : SCENE \mid store\ s = vlive\}$
 $(batchCleared = set \Rightarrow cleared' = set)$
 $(batchCleared = clear \Rightarrow cleared' = cleared)$
 $batch' = \emptyset$
 $drained' = \emptyset$
 $snapScene' = \emptyset$
 $snapVal' = vempty$
 $wphase' = wproc$
 $store' = store$
 $batchCleared' = batchCleared$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $readLock' = readLock$
 $clientLock' = clientLock$
 $shutting' = shutting$

CycleFail is the generic failure outcome: any error inside the cycle other than the two socket outcomes — a display that will not spawn, a reconnect that is refused, or any exception the two socket handlers do not catch. It can strike while the worker is sending (**wsend**) or mid-heal (**wreap**, **wensure**, **wreconn**). The drained work must not be lost, so the worker restores the *exact* drained set — every scene it drained back into **dirty**, and the clear it carried back into the live flag — releases whatever locks it held, and returns to **widle** to back off and retry. It restores **drained**, not the unsent remainder **batch**: if the batch carried a clear, the re-marked clear will blank the whole display next cycle, so every scene the batch drained — including those already sent this cycle — must be repainted, or an already-painted scene would be blanked and never redrawn. Unlike **Remark**, which re-marks every live scene for a fresh display, **CycleFail** re-marks only what this cycle drained: the display was not replaced, so only the interrupted cycle's work need be redone.

CycleFail

$\Delta Repl$

$wphase \in \{wsend, wreap, wensure, wreconn\}$
 $dirty' = dirty \cup drained$
 $(batchCleared = set \Rightarrow cleared' = set)$
 $(batchCleared = clear \Rightarrow cleared' = cleared)$
 $batch' = \emptyset$
 $drained' = \emptyset$
 $batchCleared' = clear$
 $snapScene' = \emptyset$
 $snapVal' = vempty$
 $readLock' = free$
 $clientLock' = free$
 $wphase' = widle$
 $store' = store$
 $displayed' = displayed$
 $storeLockMut' = storeLockMut$
 $mutScene' = mutScene$
 $display' = display$
 $shutting' = shutting$

9 Ending a Cycle, Clear, Shutdown

A cleared cycle blanks the display *before* it repaints the batch: the `wblank` phase runs `ApplyClear` first, and the scenes are reached only at `wproc`, so no scene is painted until the clear has been applied. The worker then repaints each batch scene from the store's current value, so a `show` that coalesced after the `clear` survives.

This is a deliberate departure from the original design text, which pushed the clear *after* the batch (for scene in batch: `push(...)`; if was_cleared: `push_clear()`). Model-checking I4 against that order found a lost update: a `clear` followed by a `show` within the worker's 16ms coalescing window painted the new scene and then blanked it (Section 11). The clear-first order is convergent for both `show-then-clear` and `clear-then-show`, and the shipped `_attempt` implements it: `clear_async` first, then the per-scene repaint. The clear's own send is modelled as an immediate blank; its send has the same time limit and the same failure outcomes as a scene send, which the send operations already cover.

When the batch is empty — every scene sent or skipped — the worker drops the batch's carried clear and goes idle.

ProcDone

$\Delta Repl$

```

wphase = wproc
batch =  $\emptyset$ 
batchCleared' = clear
wphase' = widle
store' = store
displayed' = displayed
dirty' = dirty
batch' = batch
drained' =  $\emptyset$ 
cleared' = cleared
snapScene' = snapScene
snapVal' = snapVal
storeLockMut' = storeLockMut
mutScene' = mutScene
readLock' = readLock
clientLock' = clientLock
display' = display
shutting' = shutting

```

The display fails independently of the worker: an environment step may wedge it or kill it. These are the events the worker's next send detects.

DisplayWedge

$\Delta Repl$

display = *dup*
display' = *dstuck*
store' = *store*
displayed' = *displayed*
dirty' = *dirty*
batch' = *batch*
drained' = *drained*
cleared' = *cleared*
batchCleared' = *batchCleared*
snapScene' = *snapScene*
snapVal' = *snapVal*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
clientLock' = *clientLock*
shutting' = *shutting*
wphase' = *wphase*

DisplayCrash

$\Delta Repl$

display = *dup*
display' = *ddead*
store' = *store*
displayed' = *displayed*
dirty' = *dirty*
batch' = *batch*
drained' = *drained*
cleared' = *cleared*
batchCleared' = *batchCleared*
snapScene' = *snapScene*
snapVal' = *snapVal*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
clientLock' = *clientLock*
shutting' = *shutting*
wphase' = *wphase*

At clean shutdown the worker makes one last bounded flush of whatever is still dirty and then stops. The flush is best-effort: losing the last frame is acceptable because the process is exiting. **Restart** re-enters a fresh idle state, modelling luxd starting again, and keeps the model live so the deadlock check reports only genuine lock deadlocks, never this deliberate terminal stop.

ShutdownReq

$\Delta Repl$

shutting = *clear*
shutting' = *set*
store' = *store*
displayed' = *displayed*
dirty' = *dirty*
batch' = *batch*
drained' = *drained*
cleared' = *cleared*
batchCleared' = *batchCleared*
snapScene' = *snapScene*
snapVal' = *snapVal*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
clientLock' = *clientLock*
display' = *display*
wphase' = *wphase*

Flush

$\Delta Repl$

wphase = *widle*
shutting = *set*
dirty' = $\emptyset$
wphase' = *wstopped*
store' = *store*
displayed' = *displayed*
batch' = *batch*
drained' = *drained*
cleared' = *cleared*
batchCleared' = *batchCleared*
snapScene' = *snapScene*
snapVal' = *snapVal*
storeLockMut' = *storeLockMut*
mutScene' = *mutScene*
readLock' = *readLock*
clientLock' = *clientLock*
display' = *display*
shutting' = *shutting*

Restart

$\Delta Repl$

$wphase = wstopped$
 $store' = (\lambda s : SCENE \bullet vempty)$
 $displayed' = (\lambda s : SCENE \bullet vempty)$
 $dirty' = \emptyset$
 $batch' = \emptyset$
 $drained' = \emptyset$
 $cleared' = clear$
 $batchCleared' = clear$
 $snapScene' = \emptyset$
 $snapVal' = vempty$
 $storeLockMut' = \emptyset$
 $mutScene' = \emptyset$
 $readLock' = free$
 $clientLock' = free$
 $display' = dup$
 $shutting' = clear$
 $wphase' = widle$

10 Invariants

Five properties must hold across all reachable states. None is placed in the **Repl** schema: a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant 1 — single writer. Only the worker ever holds the client send lock, and it holds it only while sending:

$$clientLock = held \Rightarrow wphase = wsend.$$

No mutator operation mentions **clientLock** in its effect, so no mutator can ever hold it. The design removes the two second-writer paths the current code has — the inline resend in the click dispatch and the beads app's direct render — so that the worker's **wsend** is the only place a send happens.

Invariant 2 — agent-path liveness. No mutator operation is guarded by **clientLock**, **display**, or **wphase**: **ReplaceBegin**, **ReplaceCommit**, **EmptyScene**, and **ClearStore** depend only on the store write lock being free. So no state of the display or of the worker's send can disable a mutator. A mutator's begin does also require **readLock = free**, the ordinary reader-writer exclusion of the store lock; but the worker holds that read lock only across **Snap**'s in-memory copy and releases it in **SendBegin** before any send, so a mutator waits at most for one snapshot copy and never for display I/O. The witness (Section 11) is a reachable state in which the worker is stuck sending to a wedged display while a mutator holds the write lock and is free to commit.

Invariant 3 — no lock cycle. No thread holds the store lock and the client send lock at once:

$$\neg (readLock = held \wedge clientLock = held).$$

The worker takes the read lock in **Snap**, releases it in **SendBegin** *before* taking the client lock, and no mutator ever takes the client lock. The two locks are therefore always acquired in one order and never held together, so no cyclic wait can form.

Invariant 4 — no lost update. Whenever the system is at rest with the display up, the display shows the store's latest value for every scene. "At rest" means the worker is idle, nothing is dirty or cleared, no mutator holds the write lock, and no shutdown is in progress:

$$Quiescent \Rightarrow (\forall s : SCENE \bullet displayed\ s = store\ s),$$

where

$$\begin{aligned} \text{Quiescent} \quad \hat{=} \quad & \text{wphase} = \text{widle} \wedge \text{dirty} = \emptyset \wedge \text{cleared} = \text{clear} \\ & \wedge \text{storeLockMut} = \emptyset \wedge \text{shutting} = \text{clear} \wedge \text{display} = \text{dup}. \end{aligned}$$

After a reap and respawn the fresh display is empty, so **Remark** re-marks every live scene; the system cannot come to rest until those scenes are pushed, which restores `displayed = store`. A generic cycle failure is no weaker: **CycleFail** puts the drained scenes back in `dirty` and re-sets the cleared flag, so the same argument holds — the system cannot be **Quiescent** while that restored work is outstanding. An emptied scene reaches rest only once its blank has been pushed (**SendOk** with `snapVal = vempty`), or, under a clear, once the whole-display blank has set it empty; either way `displayed s = store s = vempty` at rest. `batchCleared` does not appear in **Quiescent** because it is `clear` at every `widle` state: every path into `widle` — **ProcDone**, **CycleFail**, **Init**, **Restart** — resets it.

Invariant 5 — one display. A fresh display is spawned only after the old one is killed: **Ensure** is enabled only from `display = ddead`, and **Reap** is the only step that reaches `ddead` on the recovery path. So the worker never holds two live displays.

11 Verification

Type-checking is a fuzz pass:

```
fuzz docs/hub_replicator.tex
```

I3, I4, and I5 are state properties, checked by asking ProB whether their negation is *reachable*. A goal reported *not found* means the invariant holds on every reachable state:

```
# Invariant 3 (must report: goal NOT found)
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "readLock = held & clientLock = held" \
  -p DEFAULT_SETSIZE 2

# Invariant 4 (must report: goal NOT found)
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = widle & dirty = {} & cleared = clear &
        storeLockMut = {} & shutting = clear & display = dup &
        card({s | s : SCENE & displayed(s) /= store(s)}) > 0" \
  -p DEFAULT_SETSIZE 2

# Invariant 5 (must report: goal NOT found)
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wensure & display /= ddead" \
  -p DEFAULT_SETSIZE 2
```

I1 is corroborated by a reachability goal that must be *not found* — no state holds the client lock outside a send:

```
# Invariant 1 (must report: goal NOT found)
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "clientLock = held & wphase /= wsend" \
  -p DEFAULT_SETSIZE 2
```

I2 is a liveness witness that must be *found*: a reachable state in which the worker is sending to a wedged display while a mutator holds the write lock and can still commit — the agent path is not blocked by display I/O:

```
# Invariant 2 (must report: goal FOUND)
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "display = dstuck & clientLock = held & storeLockMut /= {}" \
  -p DEFAULT_SETSIZE 2
```

Deadlock freedom is the full-state-space check. `Restart` keeps the model live, so any deadlock reported would be a genuine lock deadlock:

```
# Deadlock freedom (must report: no deadlock / no counterexample)
probccli docs/hub_replicator.tex -model_check -p DEFAULT_SETSIZE 2
```

The three recovery-and-blanking behaviours added here must be reachable, or I4 would hold only because the machinery is dead. Each of these witnesses must be *found*:

```
# Generic cycle failure is reachable and restores its batch
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wsend & batch /= {}" -p DEFAULT_SETSIZE 2

# A batch that carried a clear reaches recovery (clear re-mark is live)
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wremark & batchCleared = set" -p DEFAULT_SETSIZE 2

# The empty-scene-under-clear skip is reachable
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wsnap & snapVal = vempty & batchCleared = set" \
  -p DEFAULT_SETSIZE 2

# An emptied scene is blanked without a clear (behaviour 3, happy path)
probccli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wsend & snapVal = vempty & batchCleared = clear" \
  -p DEFAULT_SETSIZE 2
```

Finding — clear ordering. Model-checking I4 against the original design document’s stated order — push the batch scenes, then if `was_cleared`: `push_clear()` — returns *goal found*. Replace the `wblank`-phase blank (which happens first) with a `PushClear` that blanks at the end of the cycle, and this trace is reachable:

```
ClearStore          ; store emptied, cleared set
ReplaceBegin/Commit s1 ; a show() lands after the clear, within
                        ; the 16ms coalescing window; store s1 = vlive
Drain               ; batch = {s1}, batchCleared = set
Snap; SendBegin; SendOk ; display now shows s1 = vlive
PushClear           ; display blanked to empty
-- rest: displayed s1 = vempty, store s1 = vlive (lost update)
```

A `clear` immediately followed by a `show` is an ordinary sequence, so this is a reachable defect, not a corner case. The clear-first order — the `wblank` phase preceding `wproc` — closes it: with clear-first the goal returns *not found*, and the shipped `_attempt` sends `clear_async` before the per-scene repaint.

Note — the socket-error paths re-mark live scenes. Both socket-error paths re-mark every live scene, and re-mark the clear the failed batch carried. `Reap/Ensure` kill and respawn, so the model resets `displayed` to empty — a fresh display shows nothing, and the re-mark of every live scene is what makes I4 hold: the system cannot come to rest until those scenes are pushed again. `Reconn` models the dead-peer path the same way, resetting `displayed` to empty on the assumption that a reconnect yields a display that must be fully repainted.

Under that assumption the clear re-mark of behaviour 2 is idempotent rather than load-bearing: a reset-to-empty display already shows no cleared scene, so re-marking the clear only reconfirms a blank that is already true. Its real purpose shows only under the opposite assumption — a reconnect that lands on the *same* still-running display, which keeps its last-received copy. There the pre-clear scene would linger, and re-marking the clear is what blanks it again. The model does not adopt that adversarial reconnect, for a reason worth stating plainly: the code’s socket-error recovery re-marks only the *live* scenes, not the full drained batch, so an *emptied* scene (store empty, not a clear) whose blank was lost to the failed send would *not* be re-queued, and on a stale-content reconnect it would linger — a reachable *Quiescent* violation the code as written does not close. The generic-failure path (`CycleFail`) restores the whole batch and so is safe; the socket-error paths are safe only if a reconnect repaints from blank.

This is recorded as the one place where the model and the code cannot be reconciled without either an adversarial-reconnect refinement that would fail I4, or a code change that restores the full batch on the socket-error paths as well.

12 Fidelity: reproducing the store race

A model that cannot reproduce the known defect proves nothing. The buggy variant is this specification with one change: the store’s reader-writer exclusion is removed from the worker’s snapshot. Concretely, delete the guard `storeLockMut = ∅` from `Snap`, so the worker can copy a scene’s value while a mutator is inside `replace_scene` and the scene is `vtorn`. The torn read then becomes reachable:

1. Mutator m runs `ReplaceBegin` on scene s : it takes the write lock and sets `store s = vtorn`, with `storeLockMut = {m}`.
2. Meanwhile the worker has a batch containing s and is at `wproc`. Without the exclusion guard, `Snap` fires: it copies `snapVal = store s = vtorn`.
3. The worker is now about to send a half-updated scene: a state with `snapVal = vtorn`.

The property that separates the two designs is therefore

```
# must be NOT found for the correct spec, FOUND for the buggy variant
probccli <spec>.tex -model_check -nodead \
    -goal "snapVal = vtorn" -p DEFAULT_SETSIZE 2
```

Under the intact exclusion the worker’s `Snap` cannot fire while any mutator holds the write lock, so `snapVal` is never `vtorn`; the reachability search returns *goal not found*. With the guard removed it returns *goal found* — the trace above. That difference is the evidence that the model detects the race the store lock was added to prevent.